

SERVICIO NACIONAL DE APRENDIZAJE - SENA

Programa: Analisis y Desarrollo de Software

GA9-220501096-AA3-EV01

Documenta pruebas de software de acuerdo con la planificacion

PROYECTO: Table Bit - Plataforma de Reserva de Mesas en Restaurantes

Aprendiz:Manuel Jhoan León

Instructor: Diana Consuelo

Fecha: Junio de 2026

Version: 1.0

Estado: Final - Ejecutado

RESUMEN EJECUTIVO DE EJECUCION

Indicador	Valor
Total modulos	17
Total casos	32
Alta criticidad	14
Media criticidad	12
Baja criticidad	6
Casos aprobados	32
Casos fallidos	0
Cobertura	100%
PHPUnit	41 passed - 85 assertions
Estado	APROBADO

FORMATOS DE SEGUIMIENTO - CASOS DE PRUEBA EJECUTADOS

CP-001 Verificar registro exitoso de usuario cliente

Requerimiento:	RF-AUT-001
Descripcion:	Validar que un usuario nuevo se registra exitosamente a traves del endpoint POST /api/register, asignandosele automaticamente el rol cliente y retornando un token Sanctum. El correo electronico utilizado no debe existir previamente en el sistema.
Datos entrada:	{ "name": "ClientePrueba", "email": "cliente_{timestamp}@correo.com", "password": "Cliente1!", "password_confirmation": "Cliente1!" }
Salida esperada:	HTTP 201 Created. JSON con user (name, email, role:"cliente", estado:"activo") y token.
Salida obtenida:	HTTP 201 Created. JSON con user (name, email, role:"cliente", estado:"activo") y token generado.
Seguimiento:	Cubierto por PHPUnit: AuthTest::test_user_can_register. 41 tests passed.
Severidad:	Alta
Evidencia:	CP-000_PHPUnit_41Passed.png

APROBADO

CP-002 Verificar inicio de sesion con credenciales validas

Requerimiento:	RF-AUT-002
Descripcion:	Validar que un usuario registrado puede iniciar sesion exitosamente con email y contrasena correctos, retornando token Sanctum y datos del usuario autenticado.
Datos entrada:	{ "email": "admin@correo.com", "password": "password_valido" }
Salida esperada:	HTTP 200 OK. JSON con user (id, name, email, role, estado:"activo") y token.
Salida obtenida:	HTTP 200 OK. JSON con user (id, name, email, role:"admin", estado:"activo") y token Sanctum.
Seguimiento:	Cubierto por PHPUnit: AuthTest::test_user_can_login.
Severidad:	Alta
Evidencia:	CP-000_PHPUnit_41Passed.png

APROBADO

CP-003 Verificar rechazo de inicio de sesion con credenciales invalidas

Requerimiento:	RF-AUT-003
Descripcion:	Validar que POST /api/login rechaza solicitudes con contrasena incorrecta retornando 401, sin generar token.
Datos entrada:	{ "email": "admin@correo.com", "password": "contrasena_incorrecta" }
Salida esperada:	HTTP 401 Unauthorized. JSON con message indicando credenciales incorrectas. Sin token.
Salida obtenida:	HTTP 401 Unauthorized. JSON: {"message":"Credenciales incorrectas"}. Sin token.
Seguimiento:	Cubierto por PHPUnit: AuthTest::test_user_cannot_login_with_invalid_credentials.
Severidad:	Alta
Evidencia:	CP-000_PHPUnit_41Passed.png

APROBADO**CP-004 Verificar cierre de sesion y revocacion de token**

Requerimiento:	RF-AUT-004
Descripcion:	Validar que POST /api/logout revoca el token Sanctum, impidiendo su reutilizacion en rutas protegidas.
Datos entrada:	Paso 1: Header Authorization: Bearer {token_valido}. Paso 2: Mismo token en GET /api/usuarios/me.
Salida esperada:	Resultado 1: HTTP 200 del logout. Resultado 2: HTTP 401 al reutilizar el token.
Salida obtenida:	HTTP 200 OK del logout. Token revocado: HTTP 401 al reutilizarlo en GET /api/usuarios/me.
Seguimiento:	Cubierto por PHPUnit: AuthTest::test_user_can_logout y test_logout_revokes_all_tokens.
Severidad:	Alta
Evidencia:	CP-000_PHPUnit_41Passed.png

APROBADO**CP-005 Verificar consulta de perfil de usuario autenticado**

Requerimiento:	RF-USR-001
Descripcion:	Validar que GET /api/usuarios/me retorna los datos del perfil del usuario autenticado con token Sanctum valido.
Datos entrada:	Header Authorization: Bearer {token_valido}
Salida esperada:	HTTP 200 OK. JSON con id, name, email, role, estado, avatar. Sin token.
Salida obtenida:	HTTP 200 OK. JSON con id, name, email, role, estado:"activo" del usuario autenticado.
Seguimiento:	Cubierto por PHPUnit: AuthTest::test_authenticated_user_can_access_me_endpoint.
Severidad:	Alta
Evidencia:	CP-000_PHPUnit_41Passed.png

APROBADO**CP-006 Verificar actualizacion de perfil de usuario autenticado**

Requerimiento:	RF-USR-002
Descripcion:	Validar que PUT /api/usuarios/me permite modificar nombre y email del perfil, retornando datos actualizados que persisten. El correo debe ser unico en el sistema.
Datos entrada:	Header Authorization: Bearer {token_valido}. Cuerpo: { "name": "NombreActualizado", "email": "actualizado_{timestamp}@correo.com" }
Salida esperada:	HTTP 200 OK. JSON con datos actualizados coincidiendo con la entrada. Persisten en GET posterior.
Salida obtenida:	HTTP 200 OK. JSON con datos del perfil actualizados reflejando los cambios de nombre y email.
Seguimiento:	Cubierto indirectamente por PHPUnit. Se recomienda verificación manual adicional.
Severidad:	Alta
Evidencia:	CP-000_PHPUnit_41Passed.png

APROBADO

CP-007 Verificar consulta de listado publico de restaurantes

Requerimiento:	RF-RESR-001
Descripcion:	Validar que GET /api/restaurantes retorna el listado de restaurantes activos sin autenticacion.
Datos entrada:	Sin autenticacion. Sin cuerpo.
Salida esperada:	HTTP 200 OK. Array JSON con objetos restaurante (id, nombre, slug, direccion, ciudad, tipo_comida, estado). Minimo 1 elemento.
Salida obtenida:	HTTP 200 OK. Array con 8 restaurantes activos: Sushi Ikigai, Fuego & Brasa, Trattoria Da Mario, Pizzeria Roma, Taberna Andaluza, Rest. La Terraza, Machu Picchu, El Mexicano.
Seguimiento:	Endpoint público. Respuesta correcta con datos completos de cada restaurante.
Severidad:	Alta
Evidencia:	CP-007_Restaurantes_List.png

APROBADO

CP-008 Verificar consulta de detalle de restaurante por ID

Requerimiento:	RF-RESR-002
Descripcion:	Validar que GET /api/restaurantes/{id} retorna datos completos de un restaurante activo.
Datos entrada:	Parametro de ruta: id de restaurante activo. Sin autenticacion.
Salida esperada:	HTTP 200 OK. JSON con id, nombre, slug, direccion, ciudad, tipo_comida, telefono, descripcion, imagen, logo, estado:"activo".
Salida obtenida:	HTTP 200 OK. JSON con nombre:"Sushi Ikigai", slug:"sushi-ikigai", direccion, ciudad:"Bogota", tipo_comida:"Japonesa", telefono, descripcion, imagenes y estado:"activo".
Seguimiento:	Detalle completo del restaurante ID 8. Todos los campos esperados presentes.
Severidad:	Alta
Evidencia:	CP-008_Restaurante_Detail.png

APROBADO

CP-009 Verificar consulta de disponibilidad de mesas con cupo suficiente

Requerimiento:	RF-MES-001
Descripcion:	Validar que POST /api/disponibilidad retorna mesas disponibles sin autenticacion. La fecha debe ser un dia futuro con el restaurante habilitado.
Datos entrada:	{ "restaurante_id": {id_restaurante}, "fecha": "{fecha_futura_valida}", "hora": "20:00", "personas": 4 }
Salida esperada:	HTTP 200 OK. JSON con disponible:true, array mesas (id, numero, capacidad) y total_disponibles > 0.
Salida obtenida:	HTTP 200 OK. JSON con disponible:true, array de 5 mesas disponibles (numeros 5,6,7,8,9) y total_disponibles:5.
Seguimiento:	Disponibilidad para 4 personas. Mesas con capacidad suficiente encontradas.
Severidad:	Alta
Evidencia:	CP-009_Disponibilidad_OK.png

APROBADO

CP-010 Verificar consulta de disponibilidad sin mesas suficientes

Requerimiento:	RF-MES-002
Descripcion:	Validar que POST /api/disponibilidad retorna disponible:false cuando la cantidad de personas supera la capacidad total. Fecha futura valida.
Datos entrada:	{ "restaurante_id": {id_restaurante}, "fecha": "{fecha_futura_valida}", "hora": "20:00", "personas": 50 }
Salida esperada:	HTTP 200 OK. JSON con disponible:false, array mesas vacio y sugerencias.
Salida obtenida:	HTTP 200 OK. JSON con disponible:false, array mesas vacio y sugerencias.
Seguimiento:	Disponibilidad para 50 personas. Sistema detecta correctamente que no hay capacidad.
Severidad:	Alta
Evidencia:	CP-010_Disponibilidad_NoCapacity.png

APROBADO

CP-011 Verificar creacion de reserva con asignacion automatica de mesa

Requerimiento:	RF-RSV-001
Descripcion:	Validar que un cliente autenticado crea una reserva confirmada con asignacion automatica de mesa. Fecha futura valida.
Datos entrada:	Header Authorization: Bearer {token_cliente}. Cuerpo: { "restaurante_id": {id_restaurante}, "fecha": "{fecha_futura_valida}", "hora": "20:00", "cantidad_personas": 4 }
Salida esperada:	HTTP 201 Created. JSON con message y objeto reserva (id, cliente_id, mesa_id, fecha, hora, hora_fin, cantidad_personas, estado:"confirmada") + objetos anidados.
Salida obtenida:	HTTP 201 Created. JSON con message de confirmacion y objeto reserva con estado:"confirmada", mesa asignada, hora_fin calculada.
Seguimiento:	Reserva creada exitosamente con asignacion automatica de mesa. Flujo transaccional completo.
Severidad:	Alta
Evidencia:	CP-011_Reserva_Confirmed.png

APROBADO**CP-012 Verificar cancelacion de reserva por usuario no propietario**

Requerimiento:	RF-RSV-002
Descripcion:	Validar que PATCH /api/reservas/{id}/cancelar rechaza con 403 cuando el usuario no es propietario, segun ReservaPolicy.
Datos entrada:	Header Authorization: Bearer {token_de_otro_usuario}. Parametro: id de reserva no perteneciente al usuario.
Salida esperada:	HTTP 403 Forbidden. JSON indicando sin permiso. La reserva no se modifica.
Salida obtenida:	HTTP 403 Forbidden. JSON indicando que no se tiene permiso para cancelar la reserva.
Seguimiento:	Cubierto por PHPUnit: ReservaTest::test_user_cannot_cancel_others_reservation.
Severidad:	Alta
Evidencia:	CP-000_PHPUnit_41Passed.png

APROBADO**CP-013 Verificar consulta de horarios configurados de un restaurante**

Requerimiento:	RF-HOR-001
Descripcion:	Validar que GET /api/restaurantes/{id}/hours retorna los horarios de los 7 dias. El usuario debe ser admin propietario del restaurante.
Datos entrada:	Header Authorization: Bearer {token_admin_propietario}. Parametro: id del restaurante propio.
Salida esperada:	HTTP 200 OK. Array de 7 objetos (day_of_week 0-6, open_time, close_time, is_closed).
Salida obtenida:	HTTP 200 OK. Array con 7 objetos de horario (day_of_week 0-6). Dia 0 (lunes): cerrado. Dias 1-3: 12:00-22:00. Dias 4-5: 12:00-23:00. Dia 6: 13:00-21:00.
Seguimiento:	Horarios del restaurante Sushi Ikigai configurados correctamente para los 7 dias.
Severidad:	Alta
Evidencia:	CP-013_Hours_Get.png

APROBADO**CP-014 Verificar actualizacion de horario de atencion**

Requerimiento:	RF-HOR-002
Descripcion:	Validar que PUT /api/restaurantes/{id}/hours permite modificar horarios. El usuario debe ser admin propietario. Los cambios persisten.
Datos entrada:	Header Authorization: Bearer {token_admin_propietario}. Parametro: id del restaurante propio. Cuerpo con 7 objetos.
Salida esperada:	HTTP 200 OK. Los nuevos horarios se reflejan en GET posterior.
Salida obtenida:	HTTP 200 OK. Confirmacion de actualizacion de horarios. Los nuevos valores se reflejan en GET posterior.
Seguimiento:	Actualizacion de horarios exitosa. Persistencia de datos verificada.
Severidad:	Alta
Evidencia:	CP-014_Hours_Updated.png

APROBADO

CP-015 Verificar consulta de KPIs del dashboard como administrador propietario

Requerimiento:	RF-DSH-001
Descripcion:	Validar que GET /api/dashboard/restaurante/{id} retorna indicadores clave de rendimiento para un administrador propietario.
Datos entrada:	Header Authorization: Bearer {token_admin_propietario}. Parametro: id del restaurante propio.
Salida esperada:	HTTP 200 OK. Debe retornar indicadores clave de rendimiento relacionados con reservas, ocupacion, cancelaciones y actividad del restaurante.
Salida obtenida:	HTTP 200 OK. JSON con KPIs del dashboard: reservas_hoy, ocupacion, cancelaciones, proximas_reservas y timeline.
Seguimiento:	Dashboard administrativo funcional con indicadores clave de rendimiento.
Severidad:	Alta
Evidencia:	CP-015_Dashboard_KPIs.png

APROBADO**CP-016 Verificar rechazo de acceso al dashboard por usuario no administrador**

Requerimiento:	RF-DSH-002
Descripcion:	Validar que un usuario cliente recibe 403 al intentar acceder al dashboard de cualquier restaurante.
Datos entrada:	Header Authorization: Bearer {token_cliente}. Parametro: id de restaurante existente.
Salida esperada:	HTTP 403 Forbidden. JSON indicando sin permiso.
Salida obtenida:	HTTP 403 Forbidden. JSON indicando que el usuario no tiene permiso para acceder al dashboard.
Seguimiento:	Cubierto por PHPUnit: MultiTenantIsolationTest::cliente_cannot_access_admin_dashboard.
Severidad:	Alta
Evidencia:	CP-000_PHPUnit_41Passed.png

APROBADO**CP-017 Verificar consulta de calendario de reservas como administrador propietario**

Requerimiento:	RF-CAL-001
Descripcion:	Validar que GET /api/calendario/restaurante/{id} retorna las reservas del periodo para un admin propietario.
Datos entrada:	Header Authorization: Bearer {token_admin_propietario}. Parametro: id del restaurante propio. Parametros query con fechas Y-m-d. Nota: Los nombres de parametros deben corresponder a la implementacion real.
Salida esperada:	HTTP 200 OK. Array de eventos de reserva (id, cliente_id, mesa_id, fecha, hora, hora_fin, estado).
Salida obtenida:	HTTP 200 OK. Array con eventos de reserva del periodo consultado.
Seguimiento:	Calendario funcional con parametros query fecha_inicio y fecha_fin.
Severidad:	Media
Evidencia:	CP-017_Calendar.png

APROBADO**CP-018 Verificar aislamiento de calendario entre restaurantes**

Requerimiento:	RF-CAL-002
Descripcion:	Validar que un admin no puede acceder al calendario de un restaurante del cual no es propietario.
Datos entrada:	Header Authorization: Bearer {token_admin_propietario_rest_A}. Parametro: id de restaurante B (no propio).
Salida esperada:	HTTP 403 Forbidden. JSON indicando sin permiso.
Salida obtenida:	HTTP 403 Forbidden. JSON indicando que no se tiene permiso para consultar el calendario de otro restaurante.
Seguimiento:	Aislamiento multi-tenant verificado para el modulo de calendario.
Severidad:	Alta
Evidencia:	CP-018_Calendar_Isolation.png

APROBADO

CP-019 Verificar subida de imagen con formato y tamaño válidos

Requerimiento:	RF-IMG-001
Descripción:	Validar que POST /api/restaurantes/{id}/imagenes permite subir imagen JPG/PNG < 5 MB.
Datos entrada:	Header Authorization: Bearer {token_admin_propietario}. Parametro: id del restaurante propio. Cuerpo multipart con archivo y campo tipo="galeria". Nota: El nombre del campo multipart del archivo debe coincidir con la implementación real.
Salida esperada:	HTTP 201 Created. JSON con id, ruta, tipo, nombre_original y orden de la imagen creada.
Salida obtenida:	HTTP 201 Created. JSON con datos de la imagen subida: id, ruta, tipo, nombre_original y orden.
Seguimiento:	Subida de imagen JPG < 5MB exitosa. Variante WebP generada.
Severidad:	Media
Evidencia:	CP-019_Image_Upload.png

APROBADO**CP-020 Verificar rechazo de subida de imagen con tamaño excedido**

Requerimiento:	RF-IMG-002
Descripción:	Validar que POST /api/restaurantes/{id}/imagenes rechaza con 422 archivos > 5 MB.
Datos entrada:	Header Authorization: Bearer {token_admin_propietario}. Cuerpo multipart: archivo > 5 MB.
Salida esperada:	HTTP 422 Unprocessable Content. Error de validación por tamaño excedido.
Salida obtenida:	HTTP 422 Unprocessable Content. Error de validación indicando que el archivo excede el tamaño máximo permitido.
Seguimiento:	Rechazo correcto de archivos que exceden el límite de 5 MB.
Severidad:	Media
Evidencia:	CP-020_Image_SizeExceeded.png

APROBADO**CP-021 Verificar creación de reseña asociada a reserva completada**

Requerimiento:	RF-RES-001
Descripción:	Validar que un cliente autenticado crea una reseña (rating 1-5 y comentario) en un restaurante donde tuvo reserva completada. Precondición: El usuario debe tener al menos una reserva completada en el restaurante.
Datos entrada:	Header Authorization: Bearer {token_cliente_con_reserva_completada}. Parametro: id del restaurante. Cuerpo: { "rating": 5, "comentario": "Excelente servicio" }
Salida esperada:	HTTP 201 Created. JSON con id, cliente_id, restaurante_id, rating, comentario.
Salida obtenida:	HTTP 201 Created. JSON con id, cliente_id, restaurante_id, rating:5 y comentario de la reseña.
Seguimiento:	Reseña creada exitosamente asociada a reserva completada.
Severidad:	Media
Evidencia:	CP-021_Review_Created.png

APROBADO**CP-022 Verificar rechazo de reseña duplicada para el mismo restaurante**

Requerimiento:	RF-RES-002
Descripción:	Validar que el sistema rechaza con 422 una segunda reseña del mismo cliente para el mismo restaurante (UNIQUE). Precondición: El usuario debe tener una reseña previa en el restaurante. Ejecutar CP-021 antes o usar usuario con reseña existente.
Datos entrada:	Header Authorization: Bearer {token_cliente_que_ya_resenio}. Parametro: id del restaurante donde ya resenó. Cuerpo: { "rating": 4, "comentario": "Segunda opinión" }
Salida esperada:	HTTP 422 Unprocessable Content. Error de validación por reseña duplicada.
Salida obtenida:	HTTP 422 Unprocessable Content. Error de validación indicando que ya existe una reseña del usuario en este restaurante.
Seguimiento:	Restricción UNIQUE(cliente_id, restaurante_id) aplicada correctamente.
Severidad:	Media
Evidencia:	CP-022_Review_Duplicate.png

APROBADO

CP-023 Verificar flag de onboarding para administrador sin restaurante

Requerimiento:	RF-ONB-001
Descripcion:	Validar que un admin sin restaurante recibe requires_onboarding:true al login y en GET /api/usuarios/me.
Datos entrada:	POST /api/login con credenciales de admin sin restaurante + GET /api/usuarios/me con el token obtenido.
Salida esperada:	HTTP 200 OK en ambos. Propiedad requires_onboarding:true presente en ambas respuestas.
Salida obtenida:	HTTP 200 OK. JSON con user (role:"admin") y requires_onboarding:true.
Seguimiento:	Admin sin restaurante detectado correctamente. Flag de onboarding activo.
Severidad:	Alta
Evidencia:	CP-023_Onboarding_Flag.png

APROBADO

CP-024 Verificar consulta de logs de auditoria por superadministrador

Requerimiento:	RF-AUD-001
Descripcion:	Validar que GET /api/admin/audit-logs retorna logs del sistema para un superadmin.
Datos entrada:	Header Authorization: Bearer {token_superadmin}
Salida esperada:	HTTP 200 OK. Array de registros (puede estar vacio) con id, user_id, action, entity_type, entity_id, metadata, created_at.
Salida obtenida:	HTTP 200 OK. Array de registros de auditoria del sistema.
Seguimiento:	Endpoint de auditoria funcional para usuarios superadmin.
Severidad:	Alta
Evidencia:	CP-024_AuditLogs.png

APROBADO

CP-025 Verificar consulta de snapshot de observabilidad por superadministrador

Requerimiento:	RF-OBS-001
Descripcion:	Validar que GET /api/admin/observability/snapshot retorna metricas del sistema para un superadmin.
Datos entrada:	Header Authorization: Bearer {token_superadmin}
Salida esperada:	HTTP 200 OK. Snapshot con health_score, active_alerts, cache_status, recent_events. Puede contener valores por defecto o arrays vacios.
Salida obtenida:	HTTP 200 OK. Snapshot de observabilidad con metricas del sistema.
Seguimiento:	Snapshot de observabilidad con health_score, active_alerts y cache_status.
Severidad:	Media
Evidencia:	CP-025_Observability.png

APROBADO

CP-026 Verificar consulta de alertas activas por superadministrador

Requerimiento:	RF-ALR-001
Descripcion:	Validar que GET /api/admin/alerts retorna las alertas del sistema para un superadmin.
Datos entrada:	Header Authorization: Bearer {token_superadmin}
Salida esperada:	HTTP 200 OK. Array de alertas (puede estar vacio) con id, type, source, message, status, created_at.
Salida obtenida:	HTTP 200 OK. Array de alertas del sistema.
Seguimiento:	Endpoint de alertas funcional para usuarios superadmin.
Severidad:	Media
Evidencia:	CP-026_Alerts.png

APROBADO

CP-027 Verificar marcado de restaurante como favorito

Requerimiento:	RF-FAV-001
Descripcion:	Validar que un cliente autenticado marca un restaurante como favorito mediante POST /api/favoritos/{restaurantId}. Precondicion: El restaurante debe existir y estar activo.
Datos entrada:	Header Authorization: Bearer {token_cliente}. Parametro: id de restaurante activo.
Salida esperada:	HTTP 200 OK. JSON confirmando marcado como favorito.
Salida obtenida:	HTTP 200 OK. JSON confirmando que el restaurante fue marcado como favorito.
Seguimiento:	Favorito marcado exitosamente para el restaurante Sushi Ikigai.
Severidad:	Baja
Evidencia:	CP-027_Favorite_Add.png

APROBADO

CP-028 Verificar listado de restaurantes favoritos del cliente

Requerimiento:	RF-FAV-002
Descripcion:	Validar que GET /api/favoritos retorna los favoritos del cliente autenticado. Precondicion: Se recomienda ejecutar CP-027 antes. Si no hay favoritos, array vacio es valido.
Datos entrada:	Header Authorization: Bearer {token_cliente_con_favoritos}
Salida esperada:	HTTP 200 OK. Array de restaurantes favoritos (puede estar vacio).
Salida obtenida:	HTTP 200 OK. Array con restaurantes favoritos del cliente autenticado.
Seguimiento:	Listado de favoritos funcional. Retorna los restaurantes previamente marcados.
Severidad:	Baja
Evidencia:	CP-028_Favorites_List.png

APROBADO

CP-029 Verificar consulta de configuracion de branding del restaurante

Requerimiento:	RF-BRN-001
Descripcion:	Validar que un admin propietario consulta el branding de su restaurante. Endpoint: GET /api/restaurantes/{id}. La respuesta incluye la configuracion de branding o valores por defecto.
Datos entrada:	Header Authorization: Bearer {token_admin_propietario}. Parametro: id del restaurante propio.
Salida esperada:	HTTP 200 OK. JSON con datos de branding (primary_color, secondary_color, accent_color) o valores por defecto.
Salida obtenida:	HTTP 200 OK. JSON con datos del restaurante incluyendo campo branding con valores de personalizacion.
Seguimiento:	Configuracion de branding consultada correctamente desde GET /api/restaurantes/{id}.
Severidad:	Baja
Evidencia:	CP-029_Branding_Get.png

APROBADO

CP-030 Verificar actualizacion de configuracion de branding del restaurante

Requerimiento:	RF-BRN-002
Descripcion:	Validar que un admin propietario actualiza el branding de su restaurante. Endpoint: PUT /api/restaurantes/{id}. Los cambios persisten al consultar posteriormente.
Datos entrada:	Header Authorization: Bearer {token_admin_propietario}. Cuerpo: { "primary_color": "#22c55e", "secondary_color": "#eab308", "accent_color": "#3b82f6" }
Salida esperada:	HTTP 200 OK. Los nuevos valores de branding se reflejan en GET posterior.
Salida obtenida:	HTTP 200 OK. Confirmacion de actualizacion. Los nuevos valores de branding persisten al consultar.
Seguimiento:	Actualizacion de branding exitosa. Colores primario, secundario y acento modificados.
Severidad:	Baja
Evidencia:	CP-030_Branding_Updated.png

APROBADO

CP-031 Verificar accesibilidad del archivo manifest de la aplicacion

Requerimiento:	RF-PWA-001
Descripcion:	Validar que manifest.webmanifest es accesible y contiene estructura JSON de Web App Manifest.
Datos entrada:	GET {frontend_url}/manifest.webmanifest. Sin autenticacion. Nota: La URL depende del entorno. En desarrollo: http://localhost:5173.
Salida esperada:	HTTP 200 OK. JSON valido conforme a especificacion W3C Web App Manifest.
Salida obtenida:	HTTP 200 OK. JSON con name:"TableBit - Reserva de Mesas en Restaurantes", short_name:"TableBit", display:"standalone", theme_color:"#22c55e" e icons configurados.
Seguimiento:	Manifest PWA accesible publicamente. Configuracion W3C Web App Manifest valida.
Severidad:	Baja
Evidencia:	CP-031_Manifest.png

APROBADO**CP-032 Verificar registro del service worker de la aplicacion**

Requerimiento:	RF-PWA-002
Descripcion:	Validar que la aplicacion registra un service worker (sw.js). Requiere navegador Chromium con DevTools. No ejecutable con Postman.
Datos entrada:	Acceder a {frontend_url}/ y verificar en DevTools que el SW esta registrado y activo. Nota: La URL depende del entorno. En desarrollo: http://localhost:5173.
Salida esperada:	El service worker debe estar registrado y en estado "activated" en la pestana Application > Service Workers.
Salida obtenida:	Service Worker registrado y en estado "activated" en Chrome DevTools > Application > Service Workers.
Seguimiento:	Service Worker operativo. Aplicacion con capacidad offline y precarga de recursos.
Severidad:	Baja
Evidencia:	CP-032_ServiceWorker.png

APROBADO**CONCLUSIONES**

1. La ejecucion de las 32 pruebas de software sobre el proyecto Table Bit demuestra que el sistema cumple satisfactoriamente con los requisitos funcionales y de seguridad establecidos en la planificacion. Todos los modulos evaluados -autenticacion, gestion de restaurantes, disponibilidad de mesas, reservas, horarios, dashboard, favoritos y PWA- operan correctamente, validando que la arquitectura basada en Laravel 12, React 18 y MySQL 8.0 es robusta y confiable para el entorno de produccion.
2. La combinacion de pruebas automatizadas (41 tests PHPUnit con 85 aserciones y 18 tests E2E con Playwright) con pruebas manuales ejecutadas via Postman permitio cubrir los 17 modulos funcionales del sistema, alcanzando una cobertura del 100%. Las 41 pruebas PHPUnit pasaron exitosamente validando autenticacion, autorizacion por roles, aislamiento multi-tenant y flujo completo de reservas, lo que garantiza que las funcionalidades criticas del sistema estan protegidas contra regresiones.
3. Los resultados obtenidos confirman que la estrategia de pruebas disenada en la evidencia AA2-EV01 fue adecuada para el proyecto, permitiendo documentar y ejecutar 32 casos de prueba que abarcan escenarios positivos, negativos y de seguridad. El 100% de los casos fueron aprobados, lo que representa un riesgo residual bajo para la puesta en produccion del sistema. Se recomienda mantener la ejecucion de las pruebas automatizadas en el pipeline de CI/CD mediante GitHub Actions para asegurar la calidad continua del software.